

practical6

Q1_Exceptions.java

```
// Import InputMismatchException: the exception thrown by Scanner when the user enters text where an integer is
// expected. Used to demonstrate exception handling.
import java.util.InputMismatchException;
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;
// Note: The other exception types shown below (ArithmeticException, ArrayIndexOutOfBoundsException, etc.) belong
// to java.lang, so they do NOT need a separate import.

/**
 * Demonstrates exception handling using try-catch for 7 common exceptions.
 * A separate try-catch block is used for each exception.
 */
public class Q1_Exceptions {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // 1. ArithmeticException: Cannot divide by zero
        // Guard the division below so the crash can be caught and reported.
        try {
            // Declare two integers, where 'b' is zero.
            int a = 10, b = 0;
            // Dividing by zero is not allowed, so this line throws ArithmeticException.
            System.out.println("Division result: " + (a / b));
        } catch (ArithmeticException e) {
            // Runs when a division by zero is attempted.
            System.out.println("ArithmeticException: Cannot divide by zero.");
        }

        // 2. ArrayIndexOutOfBoundsException: Invalid array index
        // Guard the array access below so an invalid index is handled.
        try {
            // Create an array big enough to hold only 5 elements (indexes 0 to 4).
            int[] numbers = new int[5];
            // Index 10 does not exist in the array, so this throws ArrayIndexOutOfBoundsException.
            numbers[10] = 100;
        } catch (ArrayIndexOutOfBoundsException e) {
            // Runs when an array is accessed using an invalid index.
            System.out.println("ArrayIndexOutOfBoundsException: Invalid array index.");
        }

        // 3. NumberFormatException: Marks must be numeric. Enter your roll number: abc
        // Guard the string-to-number conversion below.
        try {
            // Print a prompt asking the user to enter their roll number.
            System.out.print("Marks must be numeric. Enter your roll number: ");
            // Read the typed text as a String.
            String roll = sc.next();
            // Try to convert the String into an integer; this fails and throws
            // NumberFormatException if the user typed something like "abc".
            int rollNumber = Integer.parseInt(roll);
            // This line only runs if the conversion succeeded.
            System.out.println("Roll number accepted: " + rollNumber);
        } catch (NumberFormatException e) {
            // Runs when the string cannot be converted into a number.
            System.out.println("NumberFormatException: Marks must be numeric. Enter your roll number: abc");
        }
    }
}
```

```

}

// 4. InputMismatchException: Please enter an integer.
// Guard the nextInt() call below so wrong input types are handled.
try {
    // Print a prompt asking the user for an integer.
    System.out.print("Please enter an integer: ");
    // nextInt() throws InputMismatchException if the user enters non-numeric text.
    int value = sc.nextInt();
    // This line only runs if a valid integer was received.
    System.out.println("Integer accepted: " + value);
} catch (InputMismatchException e) {
    // Runs when the user types something that is not an integer.
    System.out.println("InputMismatchException: Please enter an integer.");
    sc.nextLine(); // clear the invalid input
}

// 5. NullPointerException: Student name is null.
// Guard the method call on the null reference below.
try {
    // Declare a String that does not point to any object (it is null).
    String studentName = null;
    // Calling .length() on a null reference throws NullPointerException.
    System.out.println("Student name length: " + studentName.length());
} catch (NullPointerException e) {
    // Runs when a method is called on a null reference.
    System.out.println("NullPointerException: Student name is null.");
}

// 6. StringIndexOutOfBoundsException: Invalid character index.
// Guard the charAt() call below so a bad index is handled.
try {
    // Store the word "Java" in a String variable.
    String word = "Java";
    // Index 10 is out of range (the string has only 4 characters),
    // so this throws StringIndexOutOfBoundsException.
    System.out.println("Character at index 10: " + word.charAt(10));
} catch (StringIndexOutOfBoundsException e) {
    // Runs when a character index is outside the string's length.
    System.out.println("StringIndexOutOfBoundsException: Invalid character index.");
}

// 7. IllegalArgumentException: Marks must be between 0 and 100.
// Guard the validation logic below so invalid marks are reported.
try {
    // Print a prompt asking the user for marks.
    System.out.print("Enter marks (between 0 and 100): ");
    // Read the marks entered by the user as an integer.
    int marks = sc.nextInt();
    // Check whether the marks are outside the allowed 0-100 range.
    if (marks < 0 || marks > 100) {
        // Marks are invalid, so throw an IllegalArgumentException with a helpful message.
        throw new IllegalArgumentException("Marks must be between 0 and 100.");
    }
    // This line only runs if the marks were valid.
    System.out.println("Marks accepted: " + marks);
} catch (IllegalArgumentException e) {
    // Runs when an argument has an illegal value (the marks were out of range).
    System.out.println("IllegalArgumentException: Marks must be between 0 and 100.");
}
}
}

```

Q2_MarksSystem.java

```
// Import InputMismatchException: the exception thrown by Scanner when the user enters text where a number (int)
// is expected. Used by the catch block.
import java.util.InputMismatchException;
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Student Marks System that accepts marks for a subject count,
 * calculates the average and demonstrates one try block with
 * multiple catch blocks.
 */
public class Q2_MarksSystem {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // One try block protects all the input and calculation statements below,
        // and each possible error is handled by its own matching catch block.
        try {
            // Print a prompt asking the user to enter the number of subjects.
            System.out.print("Enter number of subjects: ");
            // Read the number of subjects as an integer.
            int subjectCount = sc.nextInt();

            // Create an array whose size equals the number of subjects.
            int[] marks = new int[subjectCount];
            // Variable to accumulate the sum of all marks, started at 0.
            int total = 0;

            // Loop through each subject to read its marks one by one.
            for (int i = 0; i < subjectCount; i++) {
                // Print a prompt asking for the current subject's marks.
                System.out.print("Enter marks for subject " + (i + 1) + ": ");
                // Read the input as a String so it can be converted manually (demonstrates
                // NumberFormatException).
                String input = sc.next();
                // Convert the String into an integer and store it in the array at position i.
                marks[i] = Integer.parseInt(input);
                // Add the current subject's marks to the running total.
                total += marks[i];
            }

            // Calculate the average by dividing the total by the number of subjects.
            // This divides by zero (and throws ArithmeticException) if subjectCount is 0.
            double average = total / subjectCount;

            // Print a prompt asking for the index of the marks to display.
            System.out.print("Enter index to display marks for: ");
            // Read that index as an integer.
            int index = sc.nextInt();
            // Print the marks stored at that index; an invalid index throws ArrayIndexOutOfBoundsException.
            System.out.println("Marks at index " + index + ": " + marks[index]);

            // Print the final computed average.
            System.out.println("Average Marks: " + average);
        } catch (InputMismatchException e) {
            // Runs when the user types text instead of a number.
            System.out.println("InputMismatchException: User entered text instead of marks.");
            // Consume the leftover invalid input so the Scanner stays usable.
        }
    }
}
```

```
        sc.nextLine();
    } catch (NumberFormatException e) {
        // Runs when a String cannot be converted into an integer correctly.
        System.out.println("NumberFormatException: Marks are entered as a string and converted incorrectly.");
    } catch (ArithmeticException e) {
        // Runs when a mathematical error such as division by zero occurs.
        System.out.println("ArithmeticException: Cannot calculate average when the number of subjects is
zero.");
    } catch (ArrayIndexOutOfBoundsException e) {
        // Runs when the marks array is accessed using an invalid index.
        System.out.println("ArrayIndexOutOfBoundsException: Marks array accessed using an invalid index.");
    }
}
}
```

Q3_ResultSystem.java

```
// Import InputMismatchException: the exception thrown by Scanner when the user enters text where a number (int)
// is expected. Used by the outer catch block.
import java.util.InputMismatchException;
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Student Result System that accepts a student's name and marks
 * for 3 subjects using a nested try-catch block.
 */
public class Q3_ResultSystem {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Outer try-catch: Handle InputMismatchException
        try {
            // Print a prompt asking the user to enter the student's name.
            System.out.print("Enter student name: ");
            // Read the whole name line typed by the user.
            String name = sc.nextLine();
            // Print a prompt asking the user to enter the number of subjects.
            System.out.print("Enter number of subjects: ");
            // Read the number of subjects as an integer.
            int subjectCount = sc.nextInt();

            // Create an array whose size equals the number of subjects.
            int[] marks = new int[subjectCount];
            // Variable to accumulate the sum of all marks, started at 0.
            int total = 0;

            // Loop through each subject to read its marks one by one.
            for (int i = 0; i < subjectCount; i++) {
                // Print a prompt asking for the current subject's marks.
                System.out.print("Enter marks for subject " + (i + 1) + ": ");
                // Read the current subject's marks and store them in the array at position i.
                marks[i] = sc.nextInt();
                // Add the current subject's marks to the running total.
                total += marks[i];
            }

            // Inner try-catch: Handle ArithmeticException and ArrayIndexOutOfBoundsException
            try {
                // Calculate the average by dividing the total by the number of subjects.
                // This throws ArithmeticException if subjectCount is 0.
                double average = total / subjectCount;
                // Print the student's name.
                System.out.println("\nStudent: " + name);
                // Print the computed average marks.
                System.out.println("Average Marks: " + average);

                // Demonstrate ArrayIndexOutOfBoundsException
                // Loop from 0 up to AND including subjectCount, so on the last
                // iteration i equals the array size and accessing marks[i] is out of bounds.
                for (int i = 0; i <= subjectCount; i++) {
                    // Print the marks of each subject using its index.
                    System.out.println("Marks of subject " + (i + 1) + ": " + marks[i]);
                }
            } catch (ArithmeticException e) {
```

```
        // Runs when a mathematical error such as division by zero occurs.
        System.out.println("ArithmeticException: Cannot calculate average when the number of subjects is
zero.");
    } catch (ArrayIndexOutOfBoundsException e) {
        // Runs when the loop accesses an index that is outside the array.
        System.out.println("ArrayIndexOutOfBoundsException: Invalid subject index accessed.");
    }
} catch (InputMismatchException e) {
    // Runs when the user types text instead of a number somewhere inside the outer try block.
    System.out.println("InputMismatchException: User entered text instead of marks.");
    // Consume the leftover invalid input so the Scanner stays usable.
    sc.nextLine();
}
}
```

Q4_BankAccount.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;
// Note: IllegalArgumentException belongs to java.lang, so it does NOT need a separate import.

/**
 * Bank Account System that accepts the account holder's name and
 * withdrawal amount. Uses the throw statement to generate an
 * IllegalArgumentException if the amount exceeds the balance.
 */
public class Q4_BankAccount {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // The customer's available balance, started at 10000 for this demonstration.
        double balance = 10000;

        // Print a prompt asking the user to enter the account holder's name.
        System.out.print("Enter account holder's name: ");
        // Read the whole name line typed by the user.
        String name = sc.nextLine();
        // Print a prompt asking the user to enter the amount to withdraw.
        System.out.print("Enter withdrawal amount: ");
        // Read the withdrawal amount as a decimal number (double).
        double withdrawal = sc.nextDouble();

        // Guard the withdrawal logic so an invalid amount is caught and reported.
        try {
            // Check whether the requested withdrawal is more than the available balance.
            if (withdrawal > balance) {
                // The amount is too large, so throw an IllegalArgumentException with a custom message.
                throw new IllegalArgumentException("Insufficient balance");
            }
            // If no exception was thrown, deduct the withdrawal from the balance.
            balance -= withdrawal;
            // Print a message confirming how much was withdrawn and by whom.
            System.out.println(name + ", you withdrew " + withdrawal + ".");
            // Print the updated remaining balance.
            System.out.println("Remaining balance: " + balance);
        } catch (IllegalArgumentException e) {
            // Runs when the withdrawal exceeds the balance (our business-rule error).
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```